

SSAFY

Start Camp

Git

아카데미



목 차

Git

- 버전 관리 시스템
- Git의 영역
- Git의 동작
- Git 실습

참고

- 로컬
- `git init` 주의사항

Git

버전 관리 시스템



Git

분산 버전 관리 시스템



Git

분산 버전 관리 시스템



버전 관리

변화를 기록하고 추적하는 것

Google Docs 예시

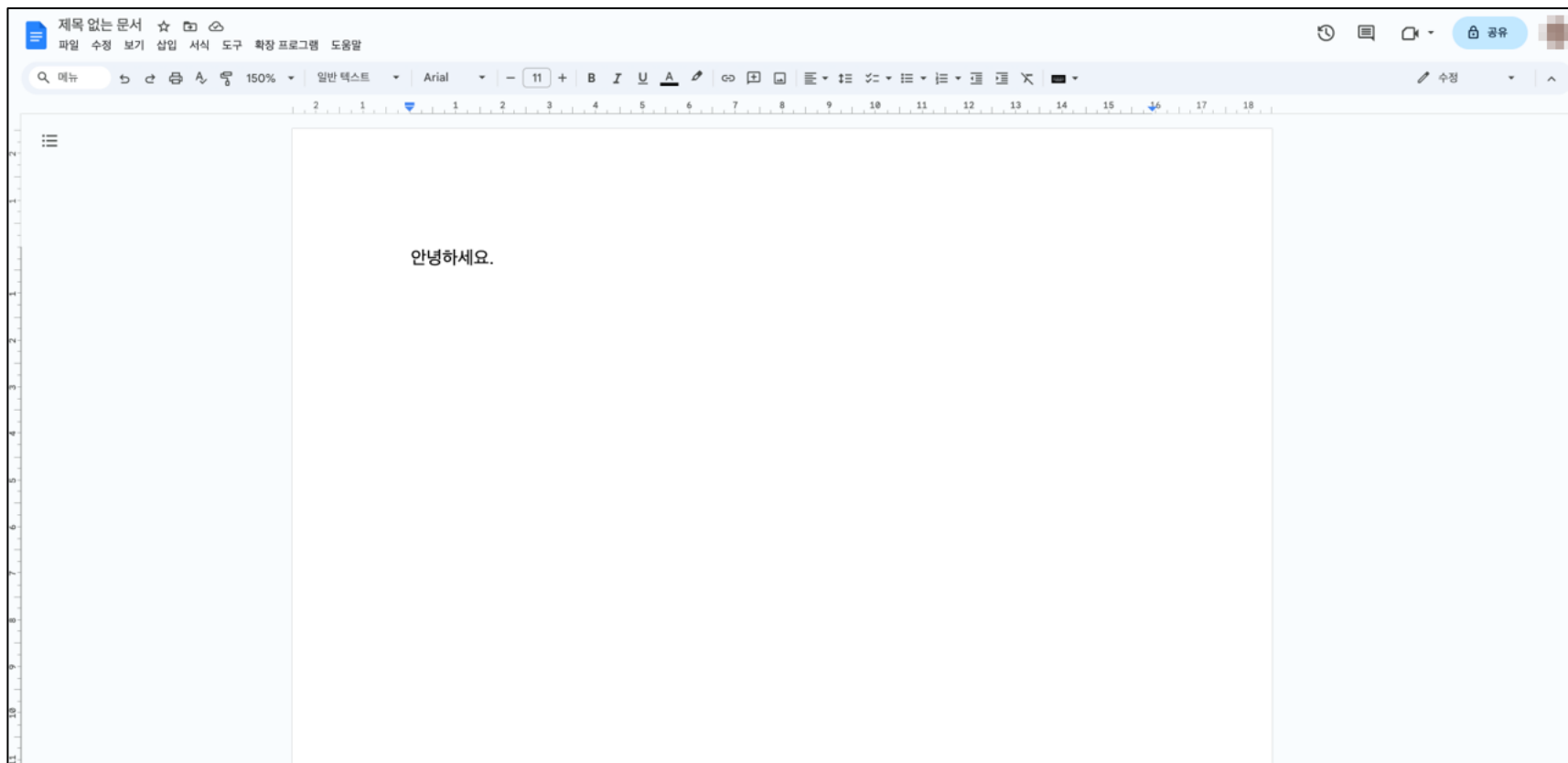


Google Docs

- Google Docs를 활용한 버전 관리 예시
 - https://www.google.com/intl/ko_KR/docs/about/

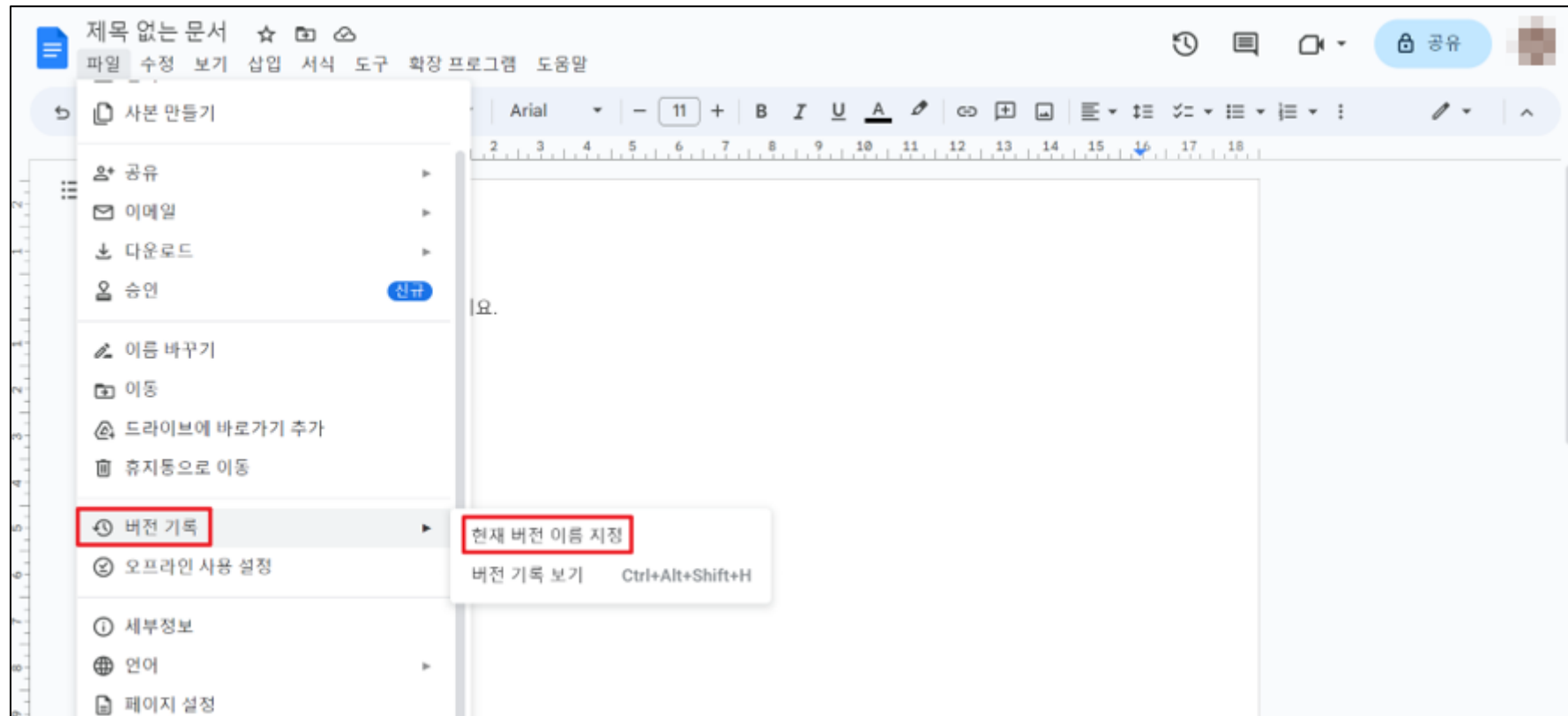
버전 관리 예시 - Google Docs (1/6)

- 임의의 텍스트 입력



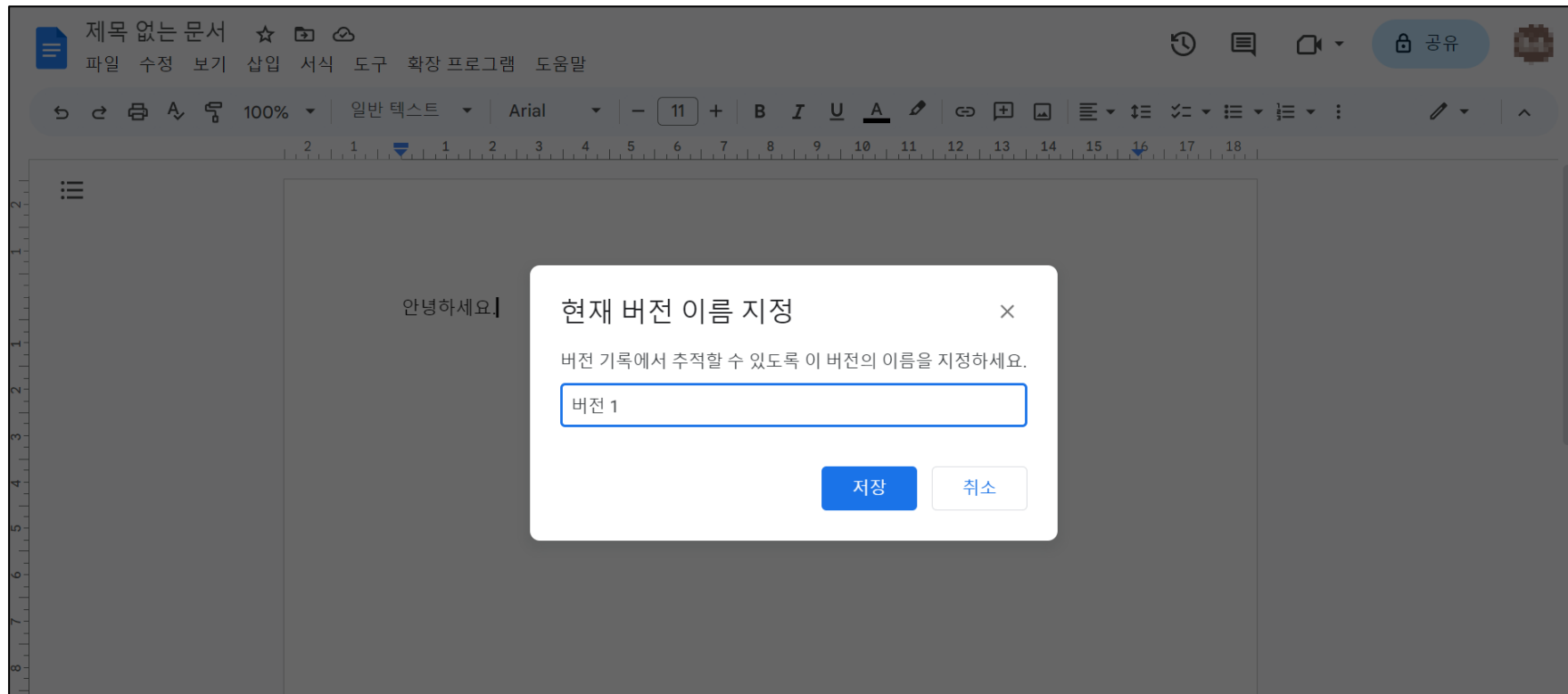
버전 관리 예시 - Google Docs (2/6)

- '파일 - 버전 기록 - 현재 버전 이름 지정' 선택



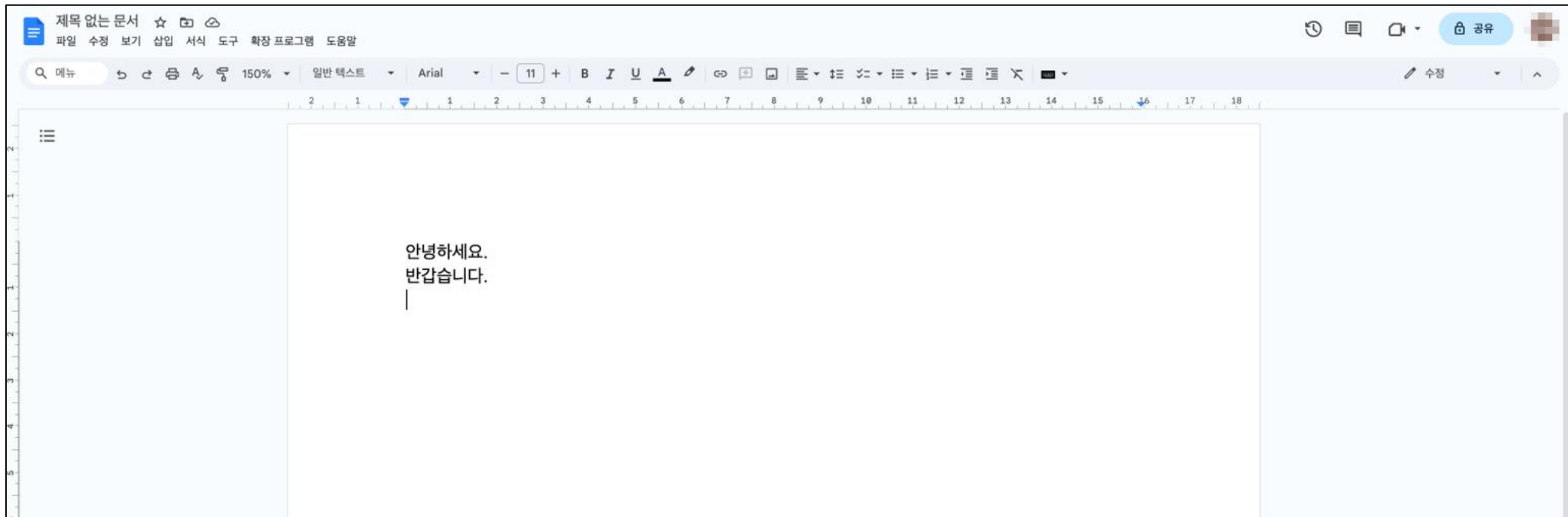
버전 관리 예시 - Google Docs (3/6)

- 현재 버전 이름 지정 후 저장



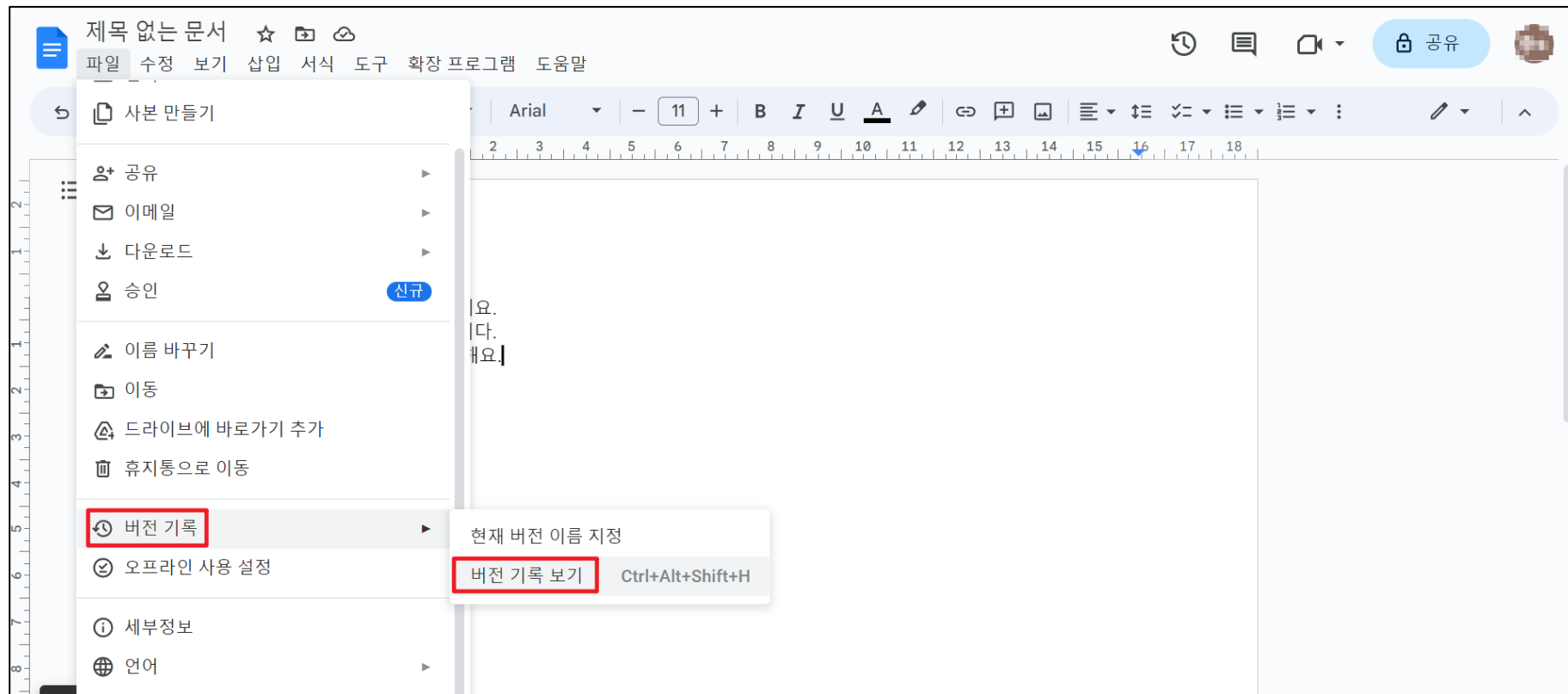
버전 관리 예시 - Google Docs (4/6)

- 추가 텍스트 입력 후 새로운 버전 입력
 - 최소 3개 이상 버전 만들기



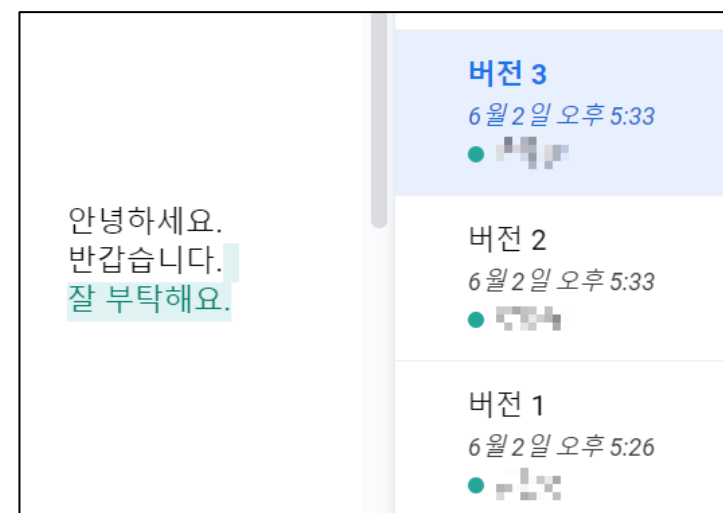
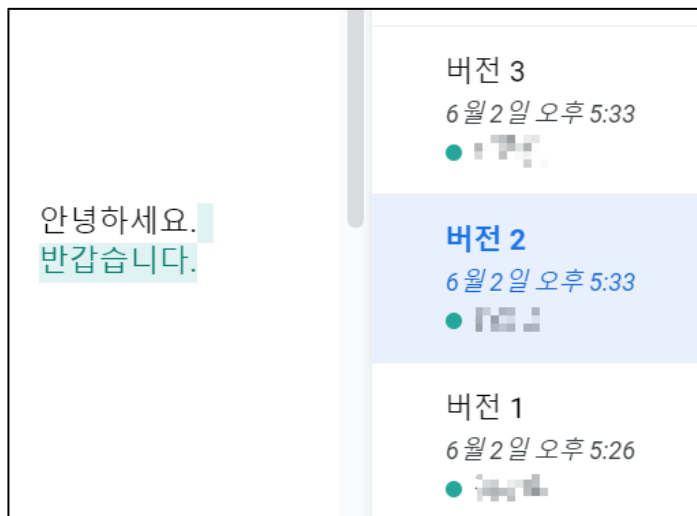
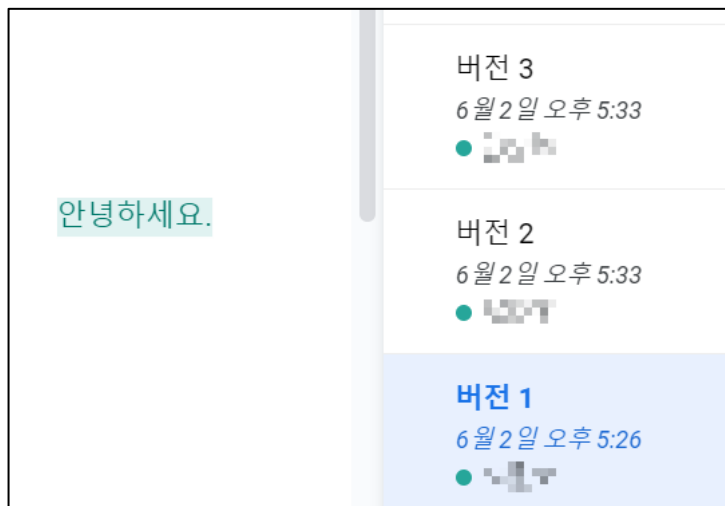
버전 관리 예시 - Google Docs (5/6)

- '파일 - 버전 기록 - 버전 기록 보기'에서 전체 버전 목록 확인

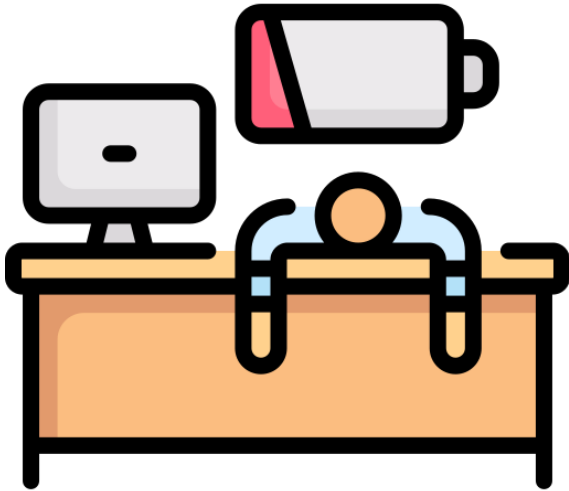


버전 관리 예시 - Google Docs (6/6)

- 각 버전은 이전 버전으로부터의 변경사항을 기록하고 있음
 - 예를 들어 버전3은 전체 내용을 모두 포함하기 있지 않음



우리는 버전 관리를 알고있다. (1/9)



졸업논문_최종.docx

졸업논문_진짜최종.docx

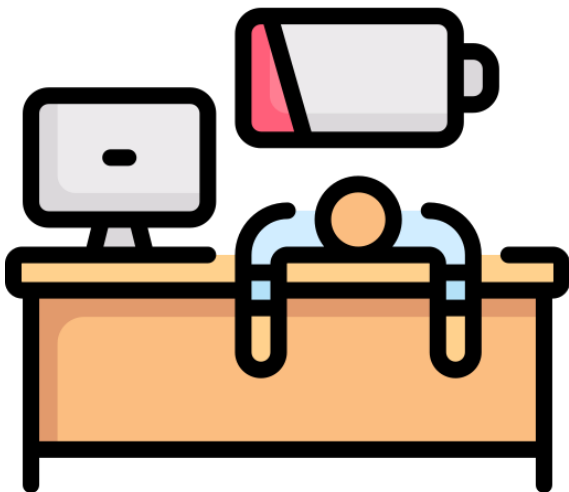
졸업논문_리얼최종.docx

졸업논문_졸업하고싶다.docx

졸업논문_진짜진짜최종.docx

졸업논문_리얼찐찐찐진짜더안함.docx

우리는 버전 관리를 알고있다. (2/9)



졸업논문_최종.docx

졸업논문_진짜최종.docx

졸업논문_리얼최종.docx

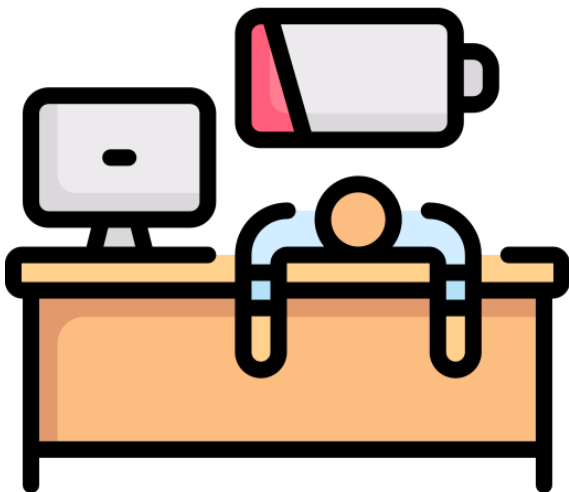
졸업논문_졸업하고싶다.docx

졸업논문_진짜진짜최종.docx

졸업논문_리얼찐찐찐진짜더안함.docx

최종 제출 파일은?

우리는 버전 관리를 알고있다. (3/9)



졸업논문_최종_0101_1604.docx

졸업논문_진짜최종 _0102_1000.docx

졸업논문_리얼최종 _0103_0530.docx

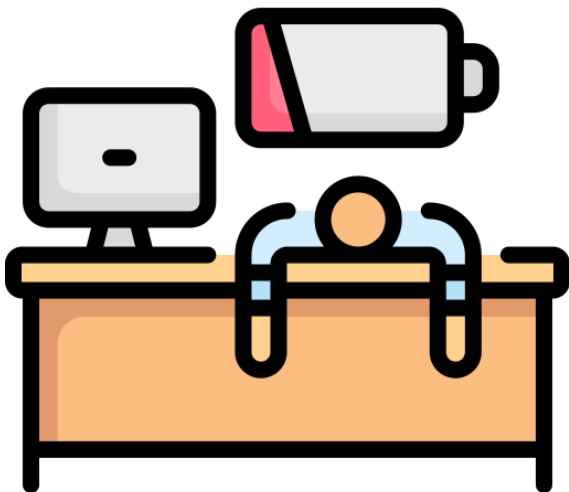
졸업논문_졸업하고싶다 _0104_0920.docx

졸업논문_진짜진짜최종 _0104_1305.docx

졸업논문_리얼찐찐찐진짜더안함 _0105_0600.docx

파일명에 날짜와 시간 기록하기

우리는 버전 관리를 알고있다. (4/9)



졸업논문_최종_0101_1604.docx

졸업논문_진짜최종 _0102_1000.docx

졸업논문_리얼최종 _0103_0530.docx

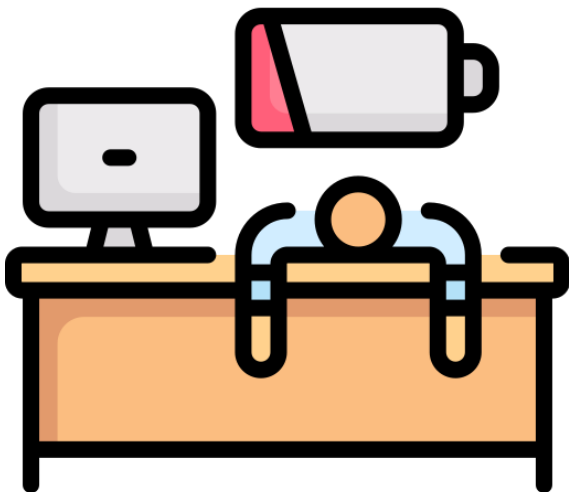
졸업논문_졸업하고싶다 _0104_0920.docx

졸업논문_진짜진짜최종 _0104_1305.docx

졸업논문_리얼찐찐찐진짜더안함 _0105_0600.docx

파일 간의 차이를 기억 할 수 없음

우리는 버전 관리를 알고있다. (5/9)



졸업논문_최종_0101_1604.docx

졸업논문_변경사항_0101_1604.docx

졸업논문_진짜최종_0102_1000.docx

졸업논문_변경사항_0102_1000.docx

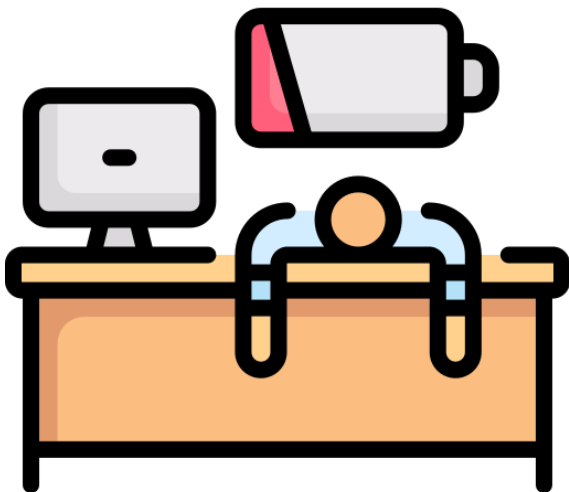
졸업논문_리얼최종_0103_0530.docx

졸업논문_변경사항_0103_0530.docx

...

변경사항을 기록하는 파일 만들기

우리는 버전 관리를 알고있다. (6/9)



만약 파일 1개당 페이지가 1000장이라면?

➤ 용량은 기하급수적으로 증가할 것

졸업논문_최종_0101_1604.docx

졸업논문_변경사항_0101_1604.docx

졸업논문_진짜최종_0102_1000.docx

졸업논문_변경사항_0102_1000.docx

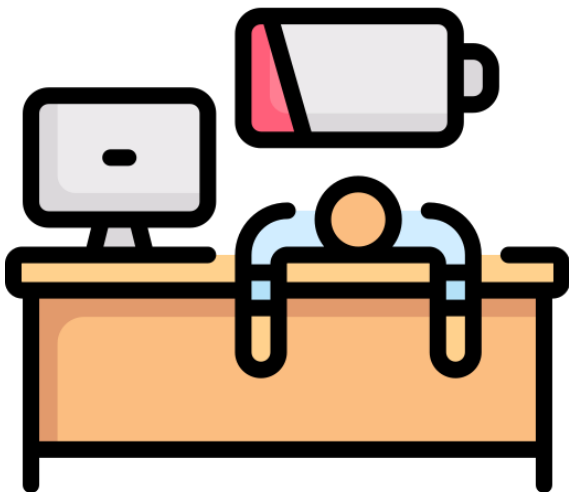
졸업논문_리얼최종_0103_0530.docx

졸업논문_변경사항_0103_0530.docx

...

변경사항을 기록하는 파일 만들기

우리는 버전 관리를 알고있다. (7/9)



졸업논문_최종_0101_1604.docx

졸업논문_변경사항_0101_1604.docx

~~졸업논문_진짜최종_0102_1000.docx~~

~~졸업논문_변경사항_0102_1000.docx~~

~~졸업논문_리얼최종_0103_0530.docx~~

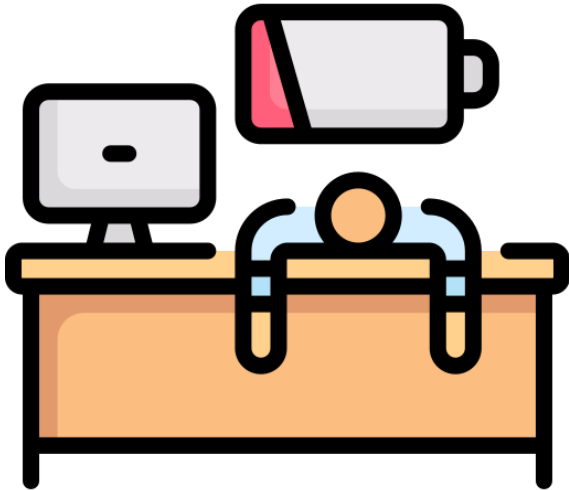
~~졸업논문_변경사항_0103_0530.docx~~

~~졸업논문_리얼찐찐찐진짜더안함_0105_0600.docx~~

~~졸업논문_변경사항_0105_0600.docx~~

마지막 파일과, 이전 변경사항만 남기기

우리는 버전 관리를 알고있다. (8/9)



졸업논문_변경사항_0101_1604.docx

졸업논문_변경사항_0102_1000.docx

졸업논문_변경사항_0103_0530.docx

졸업논문_변경사항_0104_1305.docx

졸업논문_변경사항_0105_0600.docx

졸업논문_리얼찐찐찐진짜더안함 _0105_0600.docx

최종 모습

우리는 버전 관리를 알고있다. (9/9)

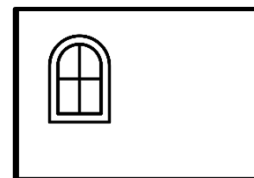
기존



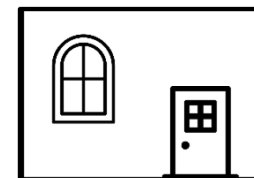
Ver1



Ver2



Ver3

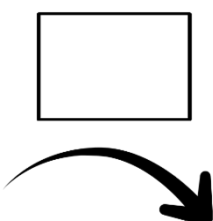


Ver4

개선



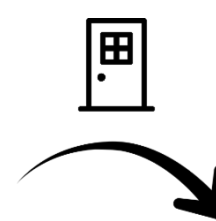
생성 전



Ver1



Ver2



Ver3



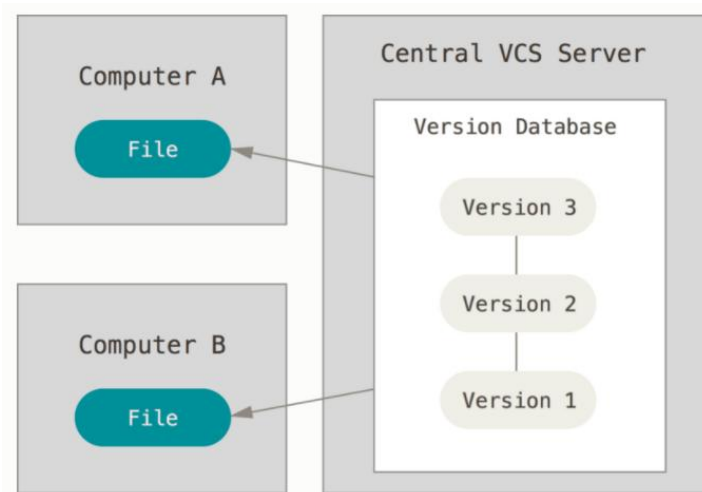
Ver4



Git

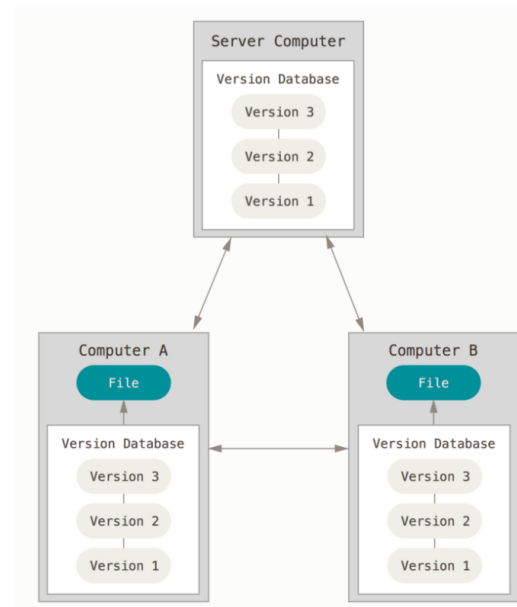
분산 버전 관리 시스템

중앙 vs 분산



중앙 집중식

버전은 중앙 서버에 저장되고
중앙 서버에서 파일을 가져와
다시 중앙에 업로드



분산식

버전을 여러 개의 복제된
저장소에 저장 및 관리

분산 구조에서의 장점

- 중앙 서버에 의존하지 않고도 동시에 다양한 작업을 수행할 수 있음
 - 개발자들 간의 작업 충돌을 줄여주고 개발 생산성을 향상
- 중앙 서버의 장애나 손실에 대비하여 백업과 복구가 용이
- 인터넷에 연결되지 않은 환경에서도 작업을 계속할 수 있음
 - 변경 이력과 코드를 로컬 저장소에 기록하고, 나중에 중앙 서버와 동기화



Git의 역할

- 코드의 버전(히스토리)을 관리
 - 개발되어 온 과정 파악
- 이전 버전과의 변경 사항 비교



Git

분산 버전 관리 시스템

- 코드의 '변경 이력'을 기록하고 '협업'을 원활하게 하는 도구

Git의 영역

Git의 3가지 영역



Working Directory



Staging Area



Repository

Git의 3가지 영역

Git의 3가지 영역

Working Directory

실제 작업 중인 파일들이 위치하는 영역

 Git의 3가지 영역

Staging Area

Working Directory에서 변경된 파일 중
다음 버전에 포함시킬 파일들을 선택적으로 추가하거나
제외할 수 있는 중간 준비 영역

 Git의 3가지 영역

Repository

버전 이력과 파일들이 영구적으로 저장되는 영역
모든 버전과 변경 이력이 기록됨

 Git의 3가지 영역

Repository

버전 이력과 파일들이 영구적으로 저장되는 영역
commit 모든 **버전**과 변경 이력이 기록됨
commit

 Commit

Commit

"버전"

변경된 파일들을 저장하는 행위이며,
마치 사진을 찍듯이 기록한다 하여 'snapshot'이라고도 함

Git의 동작

Git의 동작

- `git init`
 - 로컬 저장소 설정 (초기화)
 - git의 버전 관리를 시작할 디렉토리에서 진행
- `git add`
 - 변경사항이 있는 파일을 staging area에 추가
- `git commit`
 - staging area에 있는 파일들을 저장소에 기록
 - 해당 시점의 버전을 생성하고 변경 이력을 남기는 것

Git 사용하기 (1/12)

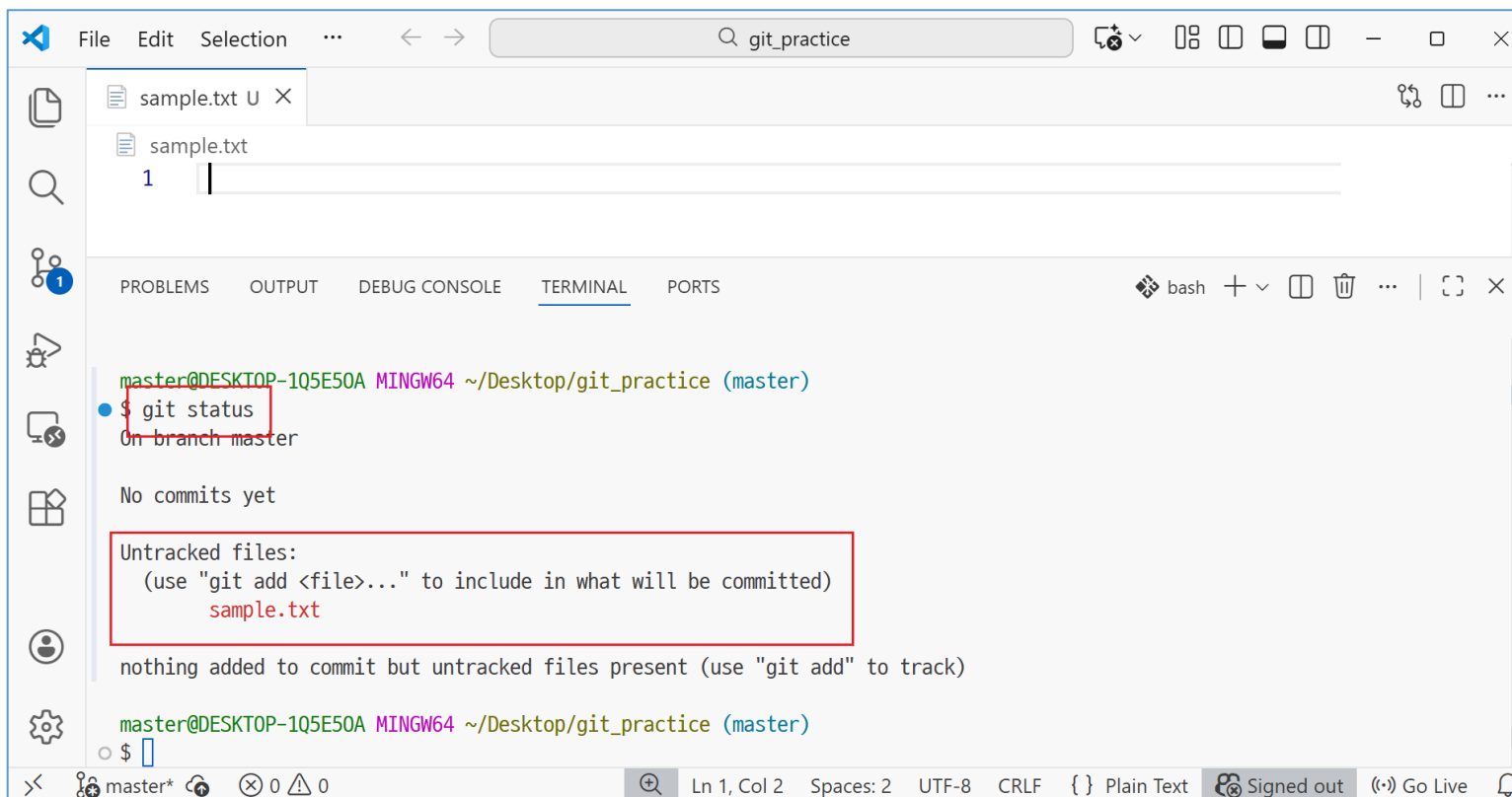
- 로컬 저장소 초기화
 - Git의 관리를 받기 시작한 디렉토리 내에서는 '(master)' 가 표시됨

```
File Edit Selection ... < > git_practice
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
master@DESKTOP-1Q5E50A MINGW64 ~/Desktop/git_practice
$ git init
Initialized empty Git repository in C:/Users/master/Desktop/git_practice/.git/
master@DESKTOP-1Q5E50A MINGW64 ~/Desktop/git_practice (master)
$
```

※ master에 대한 내용은 추후
git branch 수업에서 진행

Git 사용하기 (2/12)

- sample.txt 파일 생성 후 로컬 저장소의 파일 상태 확인



The screenshot shows the Visual Studio Code interface with a terminal window open. The terminal displays the output of the `git status` command. The output indicates that the user is on the `master` branch and that there are untracked files, specifically `sample.txt`. The command prompt shows the user is in the directory `~/Desktop/git_practice`.

```
master@DESKTOP-1Q5E50A MINGW64 ~/Desktop/git_practice (master)
$ git status
On branch master

No commits yet

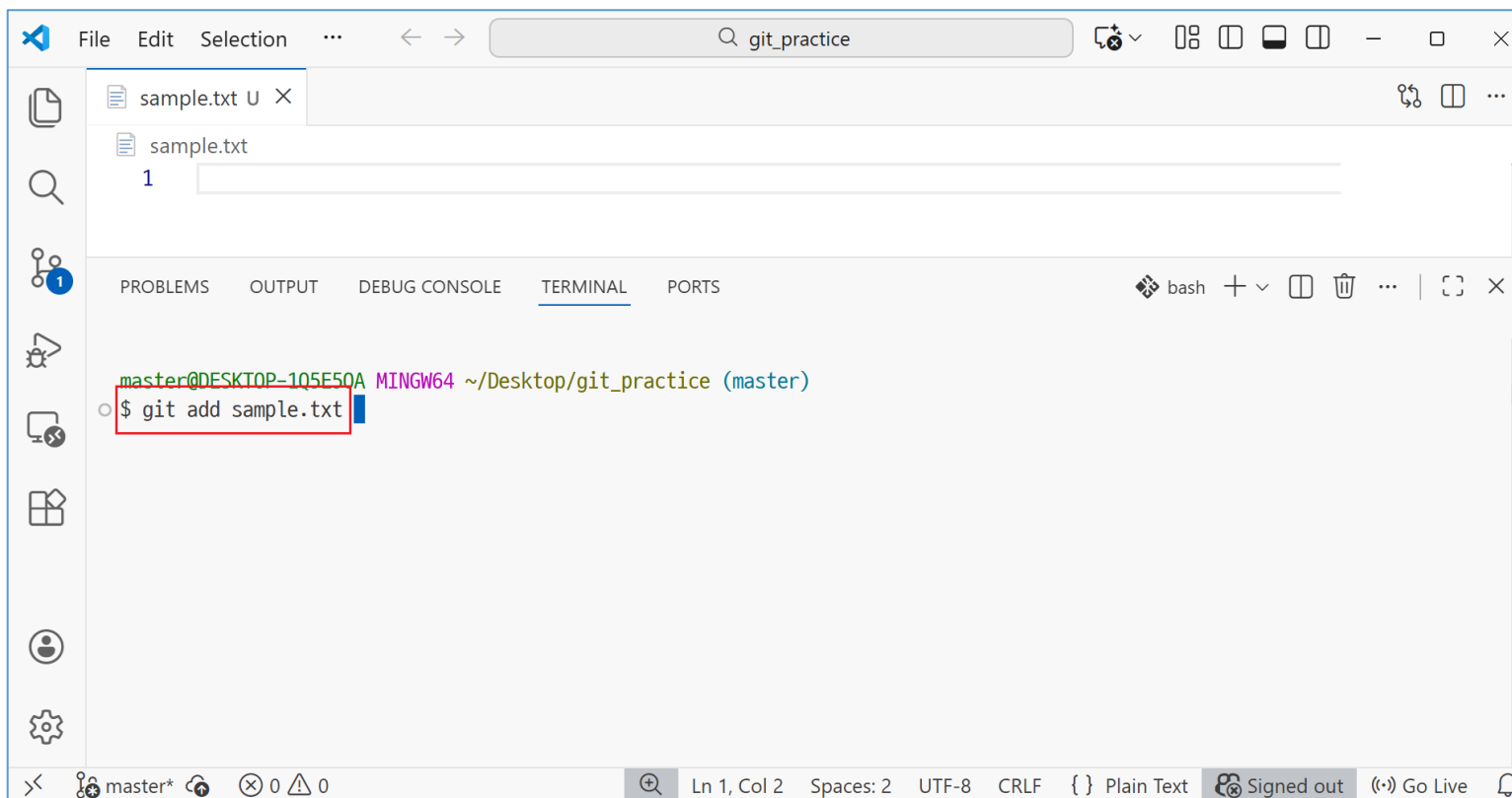
Untracked files:
  (use "git add <file>..." to include in what will be committed)
        sample.txt

nothing added to commit but untracked files present (use "git add" to track)

master@DESKTOP-1Q5E50A MINGW64 ~/Desktop/git_practice (master)
$
```

Git 사용하기 (3/12)

- sample.txt 파일을 staging area에 추가

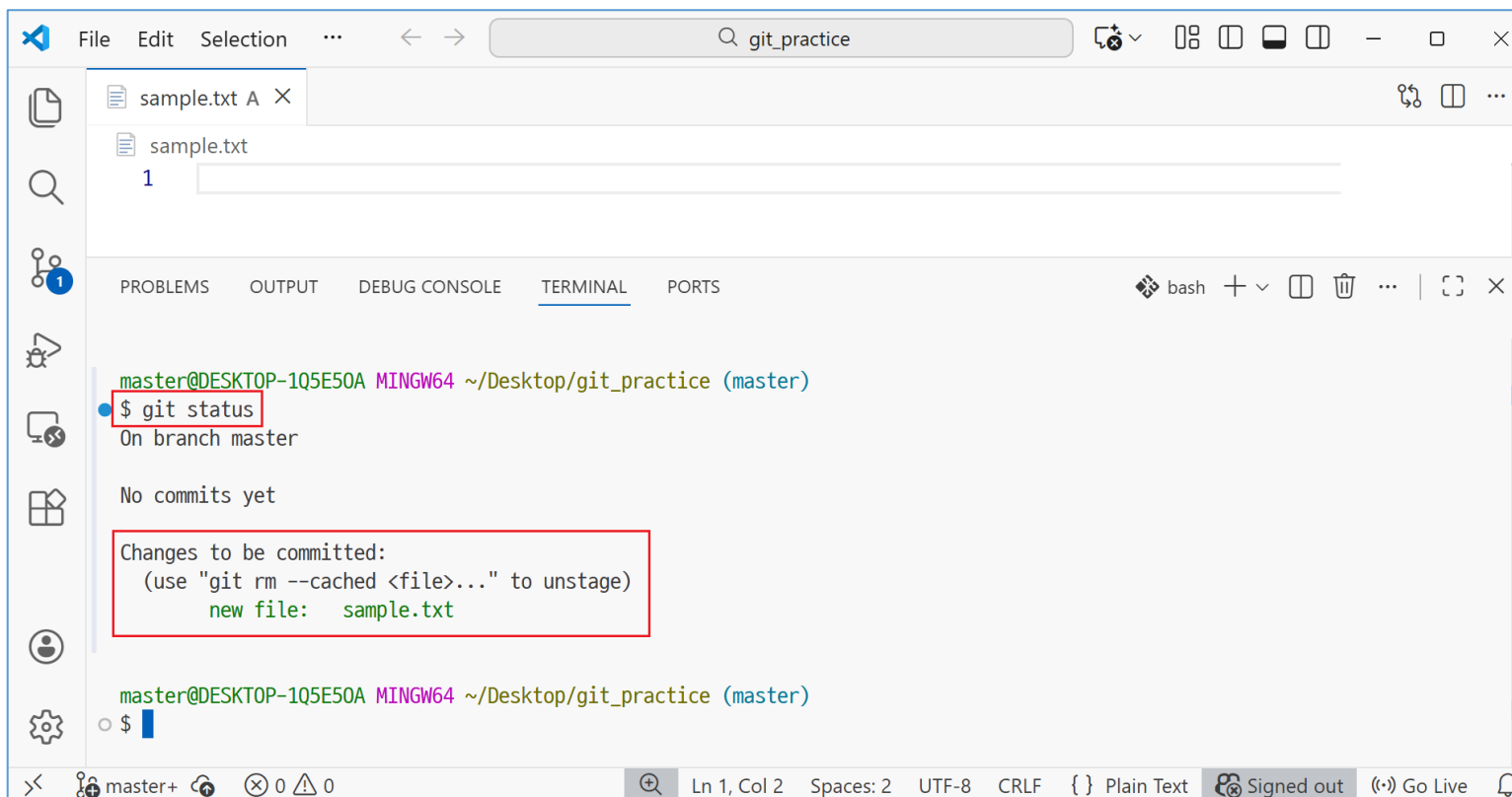


The screenshot shows the Visual Studio Code interface. The top bar displays 'File Edit Selection' and a search bar with 'git_practice'. The left sidebar shows the Explorer view with 'sample.txt' selected. The main editor area shows the content of 'sample.txt' with line 1 highlighted. The bottom panel shows the TERMINAL view with the command prompt 'master@DESKTOP-1Q5E50A MINGW64 ~/Desktop/git_practice (master)'. The command '\$ git add sample.txt' is entered and highlighted with a red box. The status bar at the bottom shows 'master*' and 'Ln 1, Col 2 Spaces: 2 UTF-8 CRLF { } Plain Text Signed out Go Live'.

```
master@DESKTOP-1Q5E50A MINGW64 ~/Desktop/git_practice (master)
$ git add sample.txt
```

Git 사용하기 (4/12)

- 로컬 저장소의 파일 상태 확인



```
File Edit Selection ... < > git_practice
sample.txt A X
sample.txt
1
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
bash
master@DESKTOP-1Q5E50A MINGW64 ~/Desktop/git_practice (master)
$ git status
On branch master

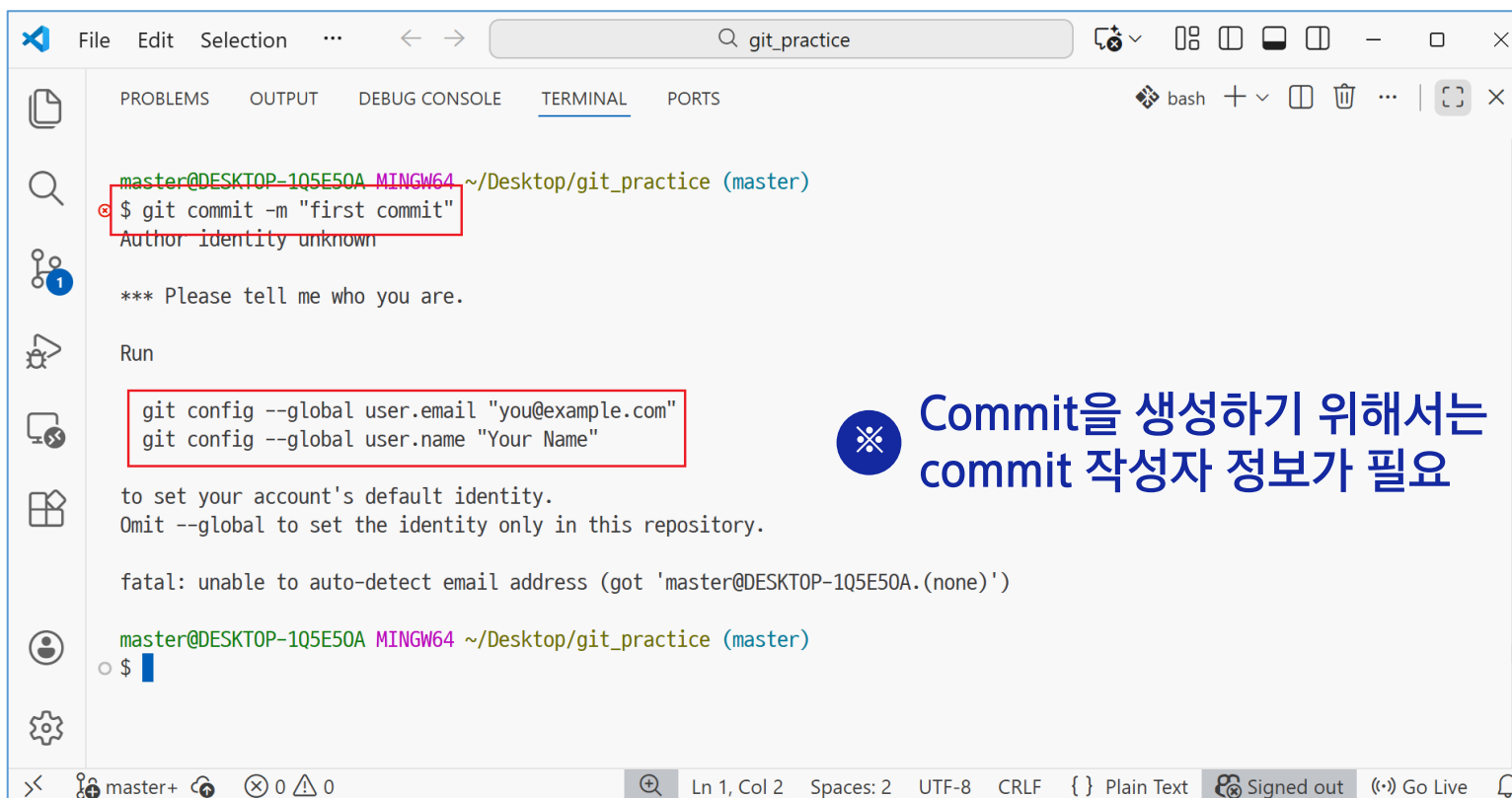
No commits yet

Changes to be committed:
  (use "git rm --cached <file>..." to unstage)
        new file:   sample.txt

master@DESKTOP-1Q5E50A MINGW64 ~/Desktop/git_practice (master)
$
```

Git 사용하기 (5/12)

- Commit 생성하기



The screenshot shows a VS Code terminal window with the following content:

```
master@DESKTOP-1Q5E50A MINGW64 ~/Desktop/git_practice (master)
$ git commit -m "first commit"
Author identity unknown

*** Please tell me who you are.

Run

git config --global user.email "you@example.com"
git config --global user.name "Your Name"

to set your account's default identity.
Omit --global to set the identity only in this repository.

fatal: unable to auto-detect email address (got 'master@DESKTOP-1Q5E50A.(none)')

master@DESKTOP-1Q5E50A MINGW64 ~/Desktop/git_practice (master)
$
```

Annotations in the image:

- A red box highlights the command `$ git commit -m "first commit"`.
- A red box highlights the error message `Author identity unknown`.
- A red box highlights the instructions for setting global git configuration: `git config --global user.email "you@example.com"` and `git config --global user.name "Your Name"`.

Overlaid text on the right side of the terminal:

✖ Commit을 생성하기 위해서는
commit 작성자 정보가 필요

 git config

```
git config --global user.email "메일주소"  
git config --global user.name "유저네임"
```

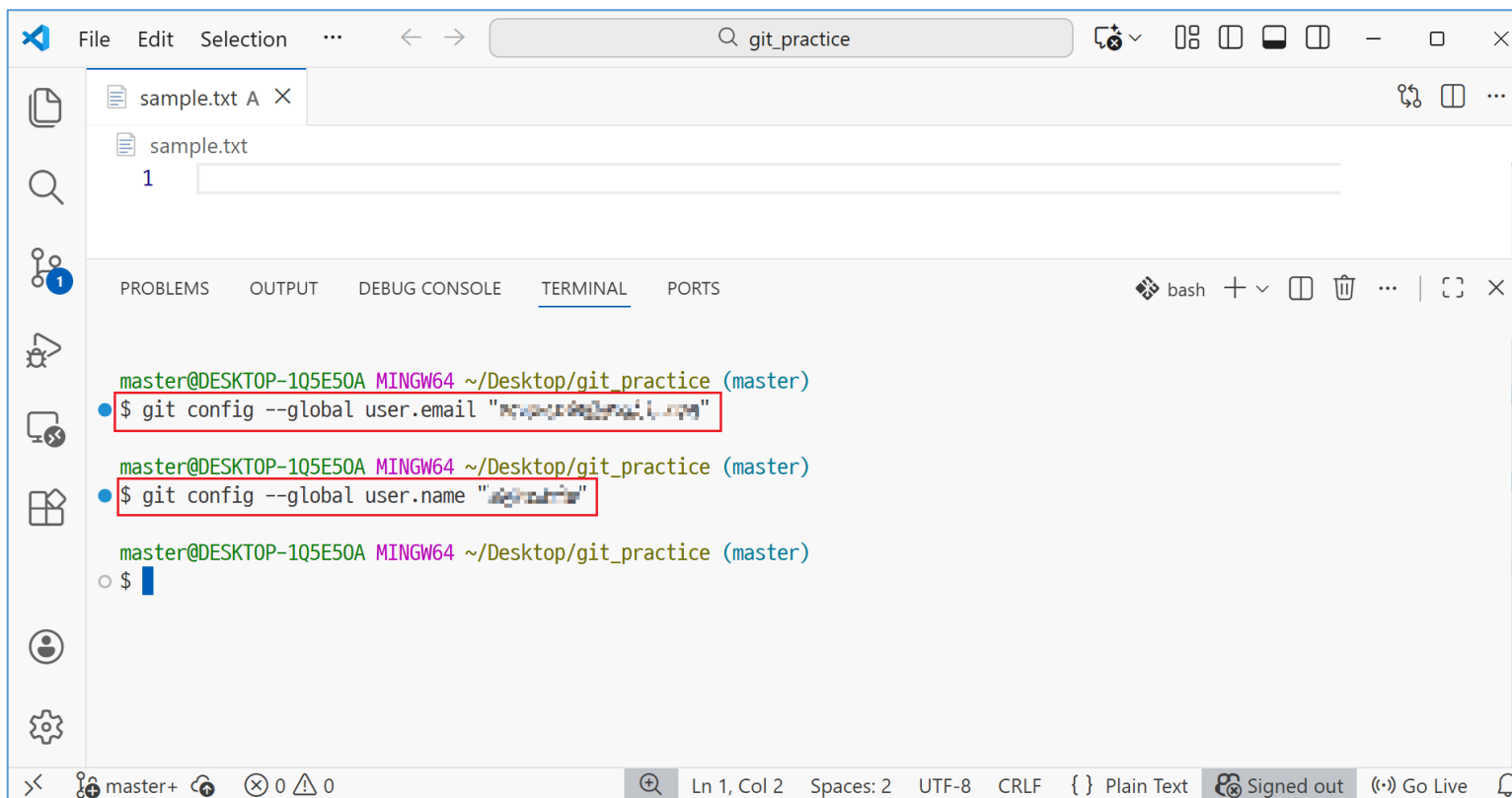
Commit 작성자(author) 정보 설정
(Global로 설정 후 앞으로 재입력하지 않음)

TIP

--global을 생략할 경우 현재 Git 저장소(현재 디렉토리의 Git 저장소)에만 적용됩니다.

Git 사용하기 (6/12)

- Git 사용자 정보 등록



The screenshot shows a Visual Studio Code window with a terminal open. The terminal is running a Windows command prompt (MINGW64) in the directory ~/Desktop/git_practice. The user is on the master branch. Two Git configuration commands are being executed and their input is highlighted with red boxes:

```
master@DESKTOP-1Q5E50A MINGW64 ~/Desktop/git_practice (master)
$ git config --global user.email "seungyeon.kim@naver.com"

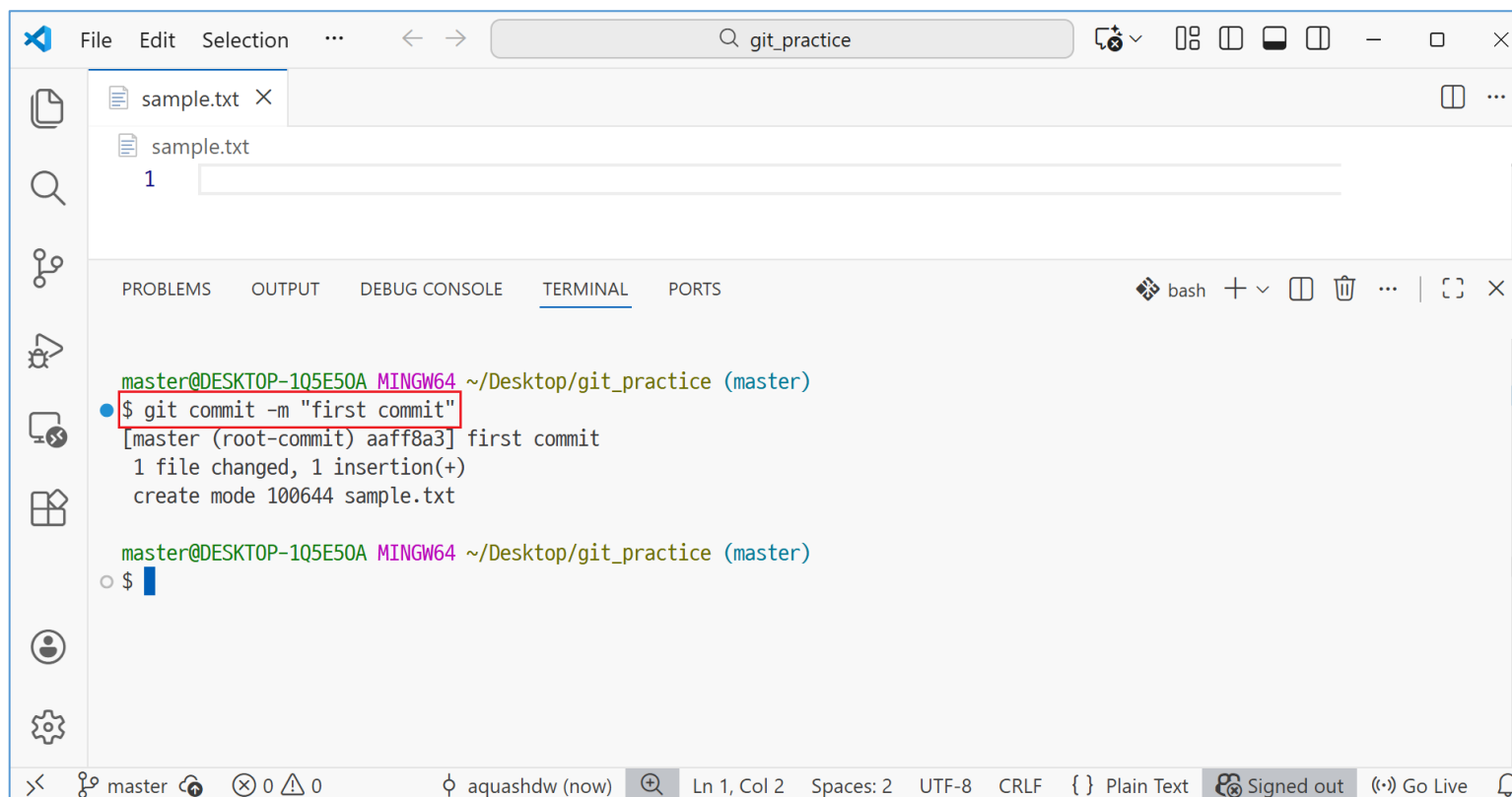
master@DESKTOP-1Q5E50A MINGW64 ~/Desktop/git_practice (master)
$ git config --global user.name "seungyeon.kim"

master@DESKTOP-1Q5E50A MINGW64 ~/Desktop/git_practice (master)
$
```

The status bar at the bottom indicates the file is signed out and shows the current line and column (Ln 1, Col 2).

Git 사용하기 (7/12)

- Commit 생성하기 재시도



The screenshot shows a Visual Studio Code editor window with a file named `sample.txt` open. The terminal panel at the bottom displays the following output:

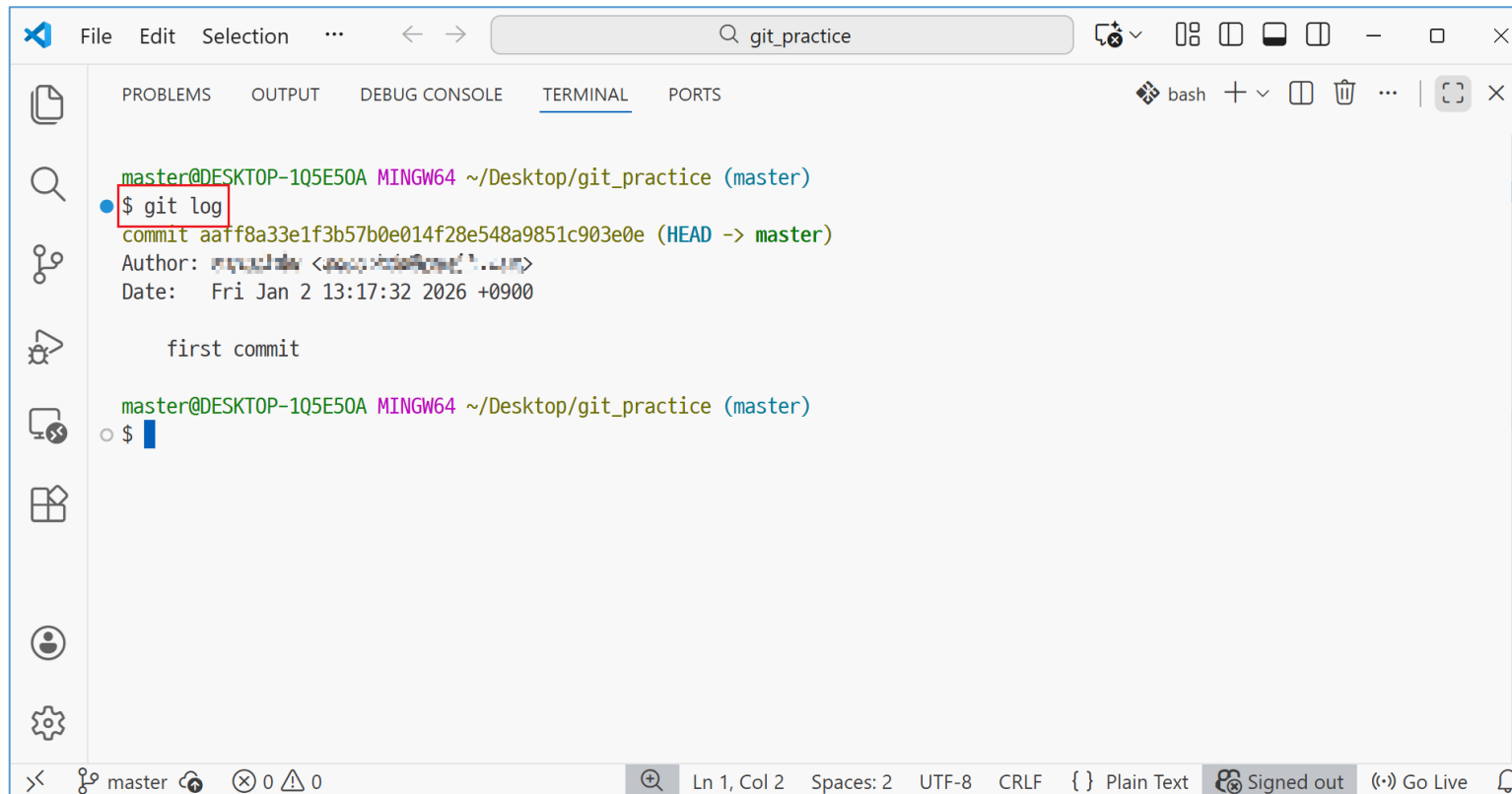
```
master@DESKTOP-1Q5E50A MINGW64 ~/Desktop/git_practice (master)
$ git commit -m "first commit"
[master (root-commit) aaff8a3] first commit
1 file changed, 1 insertion(+)
create mode 100644 sample.txt

master@DESKTOP-1Q5E50A MINGW64 ~/Desktop/git_practice (master)
$
```

The command `$ git commit -m "first commit"` is highlighted with a red box. The terminal output shows the commit was successful, creating a new root commit with hash `aaff8a3` and adding the file `sample.txt`.

Git 사용하기 (8/12)

- Commit 목록 확인



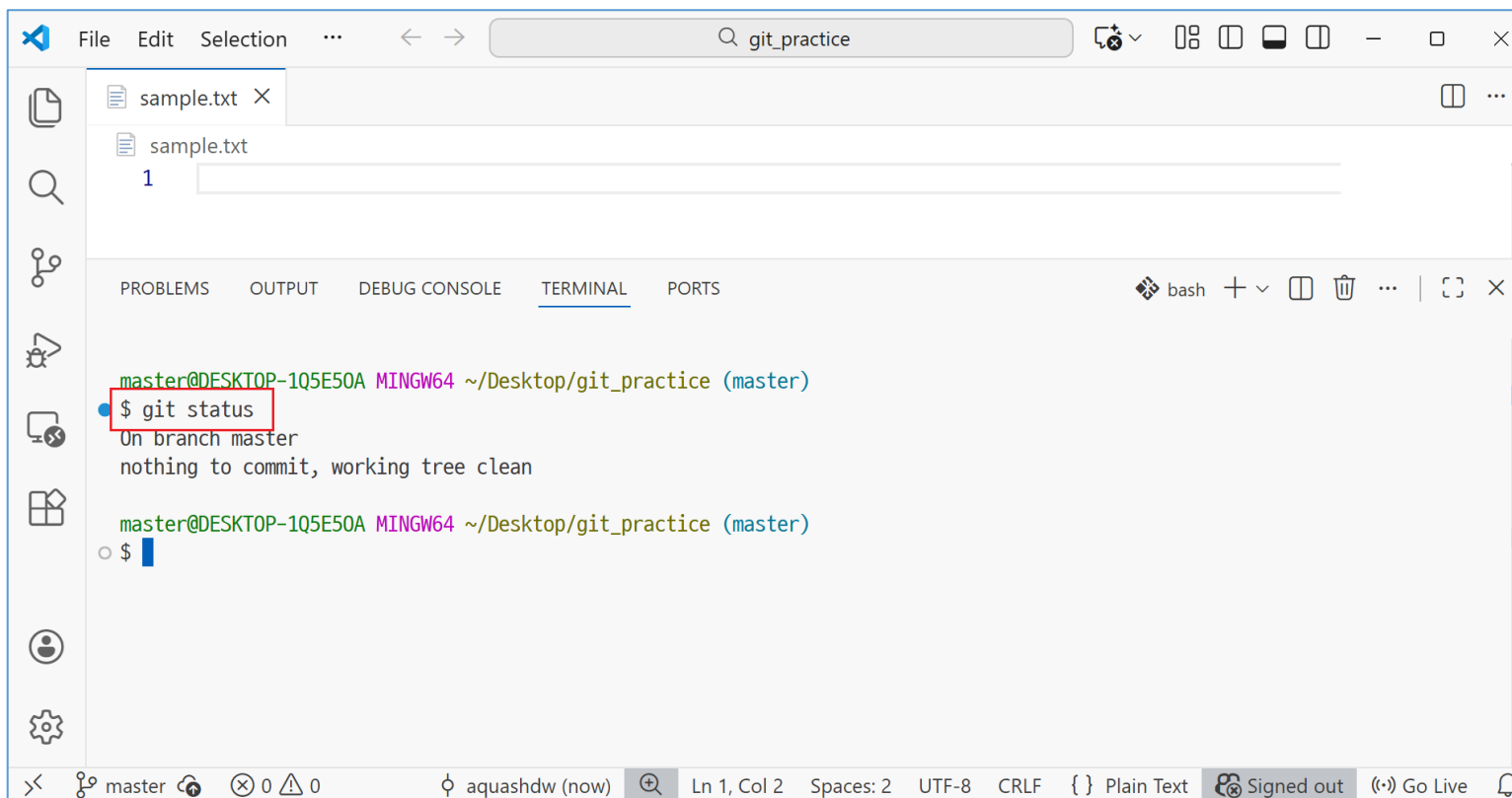
```
master@DESKTOP-1Q5E50A MINGW64 ~/Desktop/git_practice (master)
$ git log
commit aaff8a33e1f3b57b0e014f28e548a9851c903e0e (HEAD -> master)
Author: <[redacted]>
Date:   Fri Jan 2 13:17:32 2026 +0900

    first commit

master@DESKTOP-1Q5E50A MINGW64 ~/Desktop/git_practice (master)
$
```

Git 사용하기 (9/12)

- 로컬 저장소의 파일 상태 확인



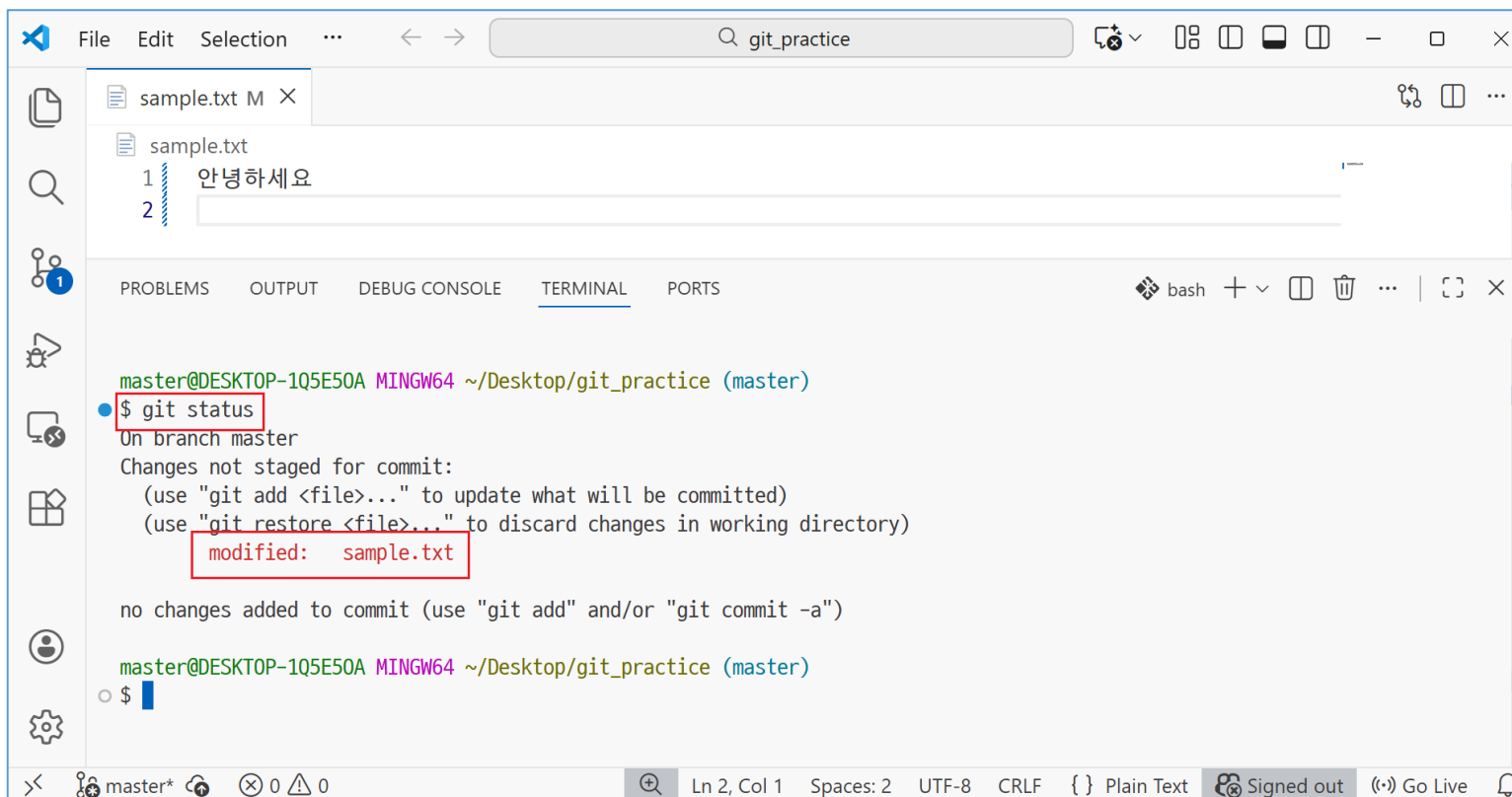
The screenshot shows the Visual Studio Code interface with a file named 'sample.txt' open. The terminal window at the bottom displays the output of the 'git status' command. The command '\$ git status' is highlighted with a red box. The output indicates that the repository is on the 'master' branch and the working tree is clean, with nothing to commit.

```
master@DESKTOP-1Q5E50A MINGW64 ~/Desktop/git_practice (master)
$ git status
On branch master
nothing to commit, working tree clean

master@DESKTOP-1Q5E50A MINGW64 ~/Desktop/git_practice (master)
$
```

Git 사용하기 (10/12)

- sample.txt에 변경사항을 만든 뒤 저장 - 로컬 저장소의 파일 상태 확인



```
File Edit Selection ... < -> git_practice
sample.txt M X
sample.txt
1 안녕하세요
2

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
bash + - - - - -

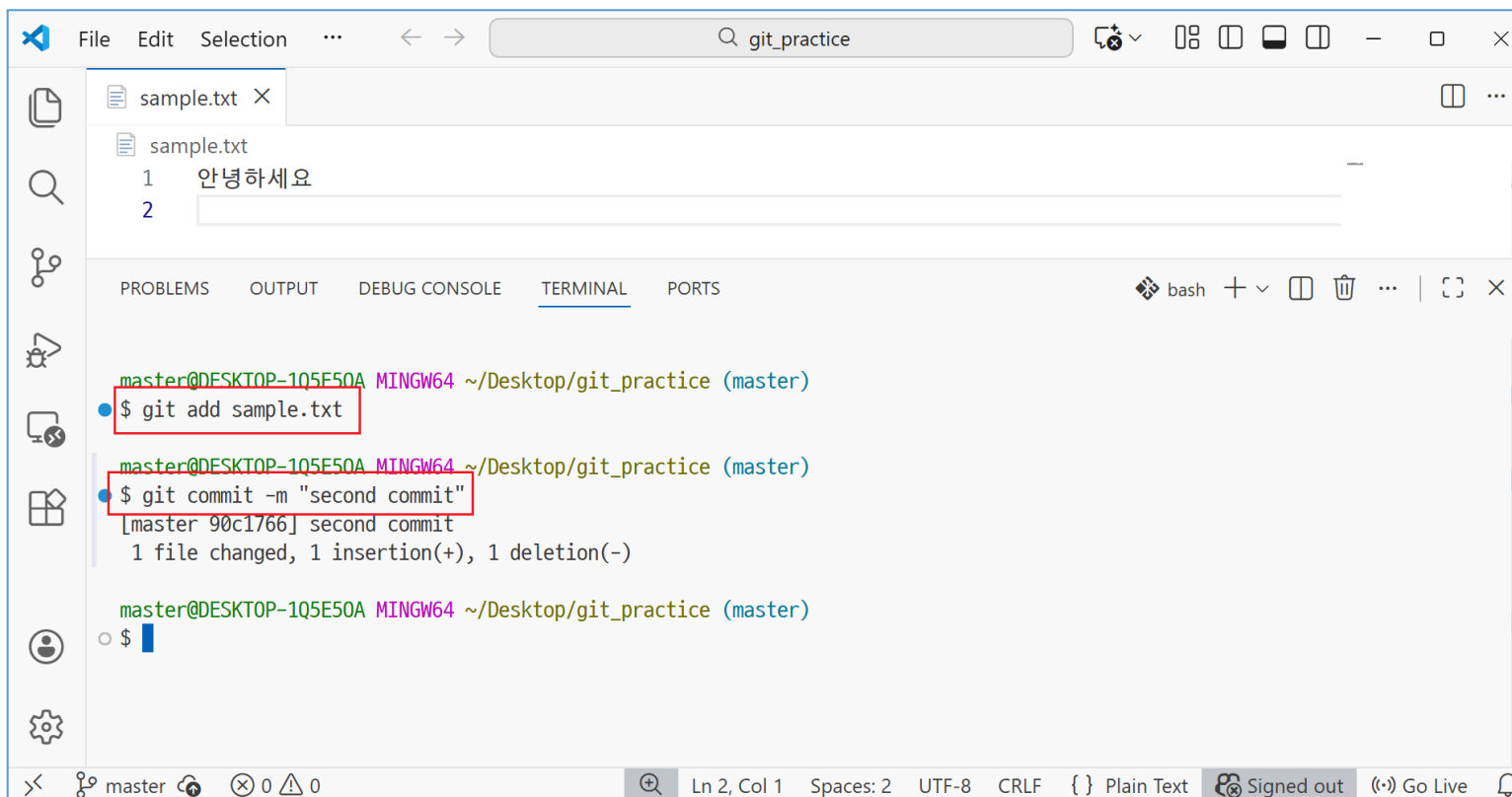
master@DESKTOP-1Q5E50A MINGW64 ~/Desktop/git_practice (master)
$ git status
On branch master
Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
        modified:   sample.txt

no changes added to commit (use "git add" and/or "git commit -a")

master@DESKTOP-1Q5E50A MINGW64 ~/Desktop/git_practice (master)
$
```

Git 사용하기 (11/12)

- 2번째 commit 생성



```
File Edit Selection ... < -> git_practice
sample.txt X
sample.txt
1 안녕하세요
2

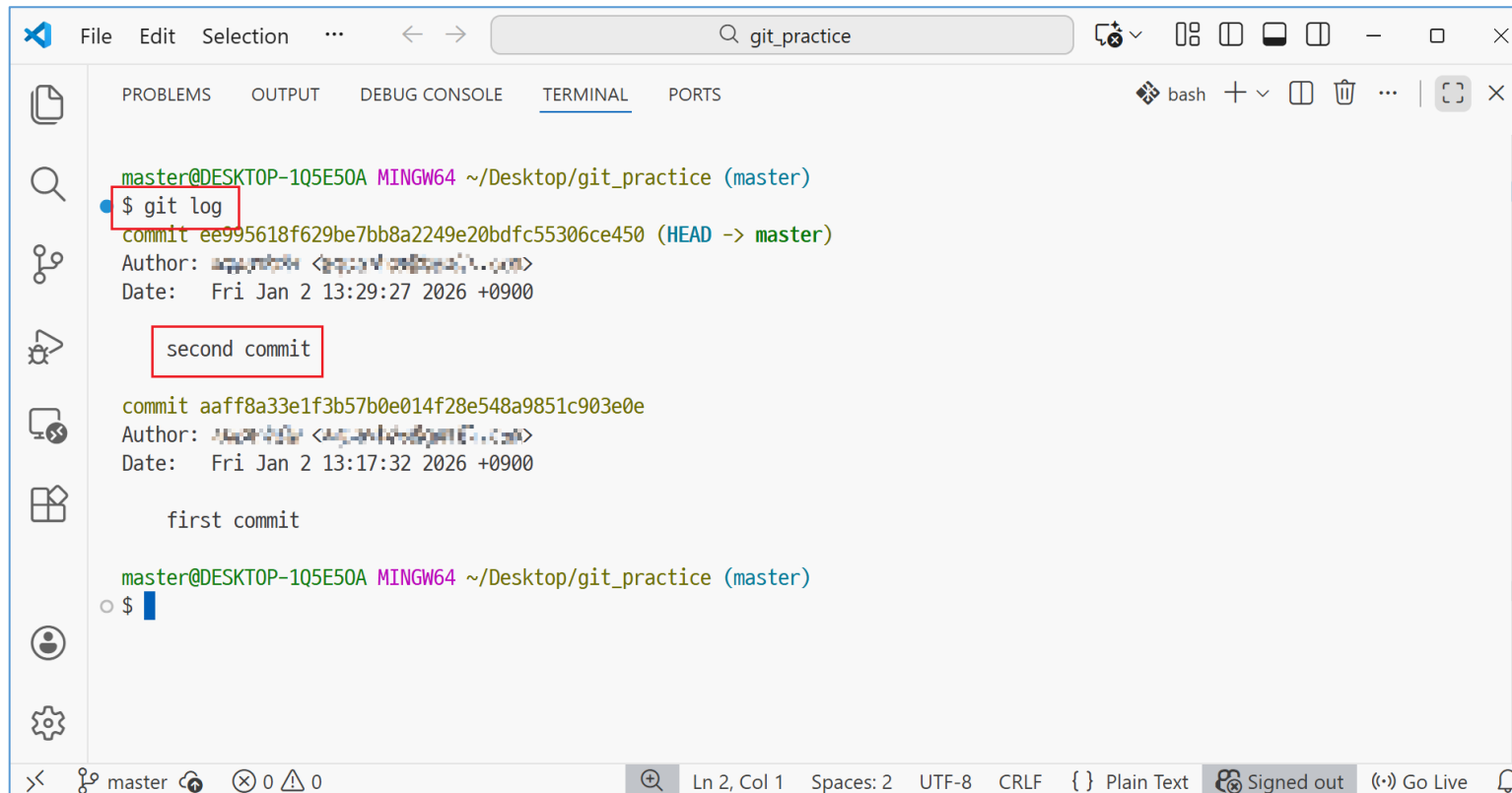
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
bash + - - - - -

master@DESKTOP-1Q5E50A MINGW64 ~/Desktop/git_practice (master)
$ git add sample.txt
master@DESKTOP-1Q5E50A MINGW64 ~/Desktop/git_practice (master)
$ git commit -m "second commit"
[master 90c1766] second commit
1 file changed, 1 insertion(+), 1 deletion(-)

master@DESKTOP-1Q5E50A MINGW64 ~/Desktop/git_practice (master)
$
```

Git 사용하기 (12/12)

- 2번째 commit 확인



```
master@DESKTOP-1Q5E50A MINGW64 ~/Desktop/git_practice (master)
$ git log
commit ee995618f629be7bb8a2249e20bdfc55306ce450 (HEAD -> master)
Author: <[redacted]>
Date: Fri Jan 2 13:29:27 2026 +0900

    second commit

commit aaff8a33e1f3b57b0e014f28e548a9851c903e0e
Author: <[redacted]>
Date: Fri Jan 2 13:17:32 2026 +0900

    first commit

master@DESKTOP-1Q5E50A MINGW64 ~/Desktop/git_practice (master)
$
```

Git의 동작

`git add sample.txt`

`git commit -m "commit 메시지"`

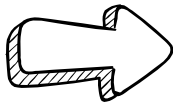
1. Working Directory

2. Staging Area

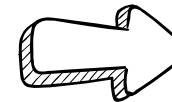
3. Repository



Untracked
Modified



New file
Modified



Nothing to commit,
working tree clean

 Git의 동작

Git은 로컬 저장소 내
모든 파일의 '변경사항'을
감시하고 있다.

기타 활용한 명령어 정리

- `git status`
 - 현재 로컬 저장소의 파일 상태 보기
- `git log`
 - Commit history 보기
- `git log --oneline`
 - Commit 목록 한 줄로 보기
- `git config --global -l`
 - Git global 설정 정보 보기

Git 실습

git commit 실습

1. 바탕화면에 git_commit 폴더를 만들고 Git 저장소 생성
2. 해당 폴더 안에 a.txt 라는 텍스트 파일을 만들고, "add a.txt" 라는 메시지로 commit 생성
3. 이번에는 b.txt 라는 텍스트 파일을 만들고, "add b.txt" 라는 메시지로 commit 생성
4. a.txt 파일을 수정하고, "update a.txt" 라는 메시지로 commit 생성
5. Commit 목록 확인하기

참고

로컬

로컬

local

현재 사용자가 직접 접속하고 있는 기기 또는 시스템
개인 컴퓨터, 노트북, 태블릿 등 사용자가 직접 조작하는 환경

git init 주의사항

git init 주의사항

- Git 로컬 저장소 내에 또다른 Git 로컬 저장소를 만들지 말 것
 - 즉, 이미 Git 로컬 저장소인 디렉토리 내부 하단에서 `git init` 명령어를 다시 입력하지 말 것
- Git 저장소 안에 Git 저장소가 있을 경우 가장 바깥 쪽의 Git 저장소가 안쪽의 Git 저장소의 변경사항을 추적할 수 없기 때문



감사합니다

